SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

Academics Strengthening & Advancement (AS&A)

Department of CSE(AI & ML)

R – 23

II B.Tech. II Semester

Learning Material

On

DATABASE MANAGEMENT SYSTEMS

UNIT – III

"SQL"

Prepared by : Mrs.G.Kalyani , Mr.N.Narayana

Dept of CSE(AI&ML)

SRGEC, Gudlavalleru

SYLLABUS

DATABASE MANAGEMENT SYSTEM

(Common to CSE,IT,AI&DS and CSE(AI&ML))

II Year - II Semester

Lecture : 3 Tutorial : 0 Internal Marks : 30
Credits : 3 External Marks : 70

Course Objectives

- To Introduce database management systems and to give a good formal foundation on the relational model of data and Relational Algebra
- To introduce the concepts of SQL for storage, retrieval and manipulation of data in a relational database.
- To demonstrate the conceptual design and logical database design through normalization
- To provide an overview of transaction management, concurrency control and indexing.

Course Outcomes:

Upon successful completion of the course, the students will be able to

- describe the importance of DBMS and derive a model in the form of ER diagram
- develop simple queries using SQL to manipulate the data in a relational model
- develop complex queries using SQL
- apply principles of normalization for designing a good relational database
- demonstrate different techniques used in transaction management and construct multi-level indices using B+ trees

UNIT I: Introduction: Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.

Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Unit II: Relational Model: Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra. BASIC SQL:Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

UNIT III: SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with

relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

UNIT IV: Schema Refinement (Normalization): Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (1NF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form(BCNF).

UNIT V:Transaction Concept: Transaction State, ACID properties, Concurrent Executions, Serializability, Recoverability, Testing for Serializability, concurrency protocols: lock based,timestamp based,-, Deadlocks, Failure Classification, Storage.
Introduction to Indexing Techniques: B+ Trees, operations on B+Trees, Hash Based Indexing

Text Books:

1. Database Management Systems, 3rd edition, Raghurama Krishnan, Johannes Gehrke, TMH (For Chapters 2, 3, 4)
2. Database System Concepts,5th edition, Silberschatz, Korth, Sudarsan,TMH (For Chapter 1 and Chapter 5)

Reference Books:

1. Introduction to Database Systems, 8thedition, C J Date, Pearson.
2. Database Management System, 6th edition, RamezElmasri, Shamkant B. Navathe, Pearson
3. Database Principles Fundamentals of Design Implementation and Management, Corlos Coronel, Steven Morris, Peter Robb, Cengage Learning.

Web-Resources:

1. https://nptel.ac.in/courses/106/105/106105175/
2. https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_0127580666728202 2456_shared/overview

UNIT – 3 Concepts of SQL

3.1 BASIC SQL QUERYING [SELECT AND PROJECT] using WHERE Clause :

- ✓ The **SELECT** statement in SQL is used to query data from a database.
- ✓ The syntax can be quite flexible depending on the complexity of your query.
- ✓ Here is the **complete syntax** of the SELECT statement in DBMS:

Complete syntax of SELECT:

```
SELECT [DISTINCT] column1, column2, ...
FROM table_names
[WHERE condition]
[GROUP BY column1, column2, ...]
[HAVING condition]
[ORDER BY column1, column2, ... [ASC|DESC]]
```

Explanation of each part:

1. **SELECT**:
   - Specifies the columns that you want to retrieve. You can select one or more columns, or even use * to select all columns.
2. **DISTINCT** (optional):
   - Used to eliminate duplicate values in the result set. Only unique rows will be returned.
3. **column1, column2, ...**:
   - Specifies the list of columns you want to retrieve data from.
4. **FROM table_name**:
   - Specifies the table from which to retrieve the data.
5. **WHERE condition** (optional):
   - Filters the records based on a specified condition. Only rows that satisfy the condition will be included in the result.
6. **GROUP BY column1, column2, ...** (optional):
   - Groups rows that have the same values in specified columns. It is typically used with aggregate functions like COUNT(), SUM(), AVG(), etc.
7. **HAVING condition** (optional):
   - Filters the grouped records (after aggregation). It is used with GROUP BY to filter groups based on a condition.
8. **ORDER BY column1, column2, ... [ASC|DESC]** (optional):
   - Sorts the result set based on one or more columns. You can specify ascending (ASC, which is default) or descending (DESC) order.

Sample Employees table:

employee_id	first_name	last_name	salary	department
1001	John	Smith	60000	Sales
1002	Jane	Doe	75000	Marketing
1003	Alice	Johnson	82000	Engineering
1004	Bob	Brown	54000	Human Resources
1005	Clara	Wilson	98000	Engineering
1006	David	Lee	67000	Sales
1007	Emma	Davis	72000	Marketing
1008	Frank	Moore	50000	Support
1009	Grace	Taylor	88000	Engineering
1010	Henry	Miller	59000	Support

Example 1: Simple SELECT query

```
SELECT first_name, last_name   FROM employees;
```

Output:

first_name	last_name
John	Smith
Jane	Doe
Alice	Johnson
Bob	Brown
Clara	Wilson
David	Lee
Emma	Davis
Frank	Moore
Grace	Taylor
Henry	Miller

Example 2: SELECT with WHERE clause

```
SELECT first_name, last_name, salary FROM employees  WHERE salary > 50000;
```

Output:

first_name	last_name	salary
John	Smith	60000
Jane	Doe	75000
Alice	Johnson	82000
Bob	Brown	54000
Clara	Wilson	98000
David	Lee	67000
Emma	Davis	72000
Grace	Taylor	88000
Henry	Miller	59000

Example 3: SELECT with DISTINCT

```
SELECT DISTINCT department FROM employees;
```

Output:

department
Sales
Marketing
Engineering
Human Resources
Support

Example 4: SELECT with GROUP BY and HAVING

```
SELECT department, COUNT(*) AS employee_count FROM employees
GROUP BY department HAVING COUNT(*) > 1;
```

Output:

department	employee_count
Engineering	3
Marketing	2
Sales	2
Support	2
Marketing	2

Example 5: SELECT with ORDER BY

SELECT last_name FROM employees ORDER BY salary DESC;

Output:

last_name
Wilson
Taylor
Smith
Moore
Miller
Lee
Johnson
Doe
Davis
Brown

Note:

- ✓ order by column by using column number also
- ✓ by default order by is ascending order

Example 6: SELECT with LIMIT/ROWNUM

SELECT first_name, last_name FROM employees WHERE ROWNUM<=5;

Output:

first_name	last_name
John	Smith
Jane	Doe
Alice	Johnson
Bob	Brown
Clara	Wilson

Example 7: SELECT with multiple conditions in WHERE

SELECT * FROM employees WHERE salary > 65000 AND department = 'Sales';

Output:

employee_id	first_name	last_name	Salary	department
1006	David	Lee	67000	Sales

3.2 Arithmetic and logical operations:

In SQL, you can perform **arithmetic** and **logical operations** to manipulate data and filter records. Here's a breakdown of both types of operations:

1. Arithmetic Operations in SQL

Arithmetic operations allow you to perform calculations on numeric data. The basic arithmetic operators in SQL are:

- **+** (Addition)
- **-** (Subtraction)
- ***** (Multiplication)
- / (Division)
- **%** (Modulo – remainder of division)

Example of Arithmetic Operations:

Let's say we have a table sales with columns price and quantity.

PRICE	QUANTITY
19.99	2
5.49	10
12	5
7.25	8
49.99	1
3.5	20
15	3
100	1
9.99	6
2.99	15

Example:

```
SELECT price, quantity, price * quantity AS total_sales  FROM sales;
```

This query calculates the total sales by multiplying price and quantity.

Output:

PRICE	QUANTITY	TOTAL_SALES
19.99	2	39.98
5.49	10	54.9
12	5	60
7.25	8	58
49.99	1	49.99
3.5	20	70
15	3	45
100	1	100
9.99	6	59.94
2.99	15	44.85

Example 2: Adding a value to a column

```
SELECT product_name, price, price + 10 AS new_price
FROM products;
```

This query increases the price by 10 and displays the updated price as new_price.

Example 3: Using division and modulo

```
SELECT order_id, quantity, quantity / 2 AS half_quantity, quantity % 2 AS remainder
FROM orders;
```

This query calculates half of the quantity and also returns the remainder when quantity is divided by 2.

2. Logical Operations in SQL

Logical operations in SQL are used to filter records based on boolean logic. The basic logical operators in SQL are:

- **AND**: Combines two conditions; both must be true for the record to be included.
- **OR**: Combines two conditions; at least one must be true for the record to be included.
- **NOT**: Reverses the truth value of a condition.
- **BETWEEN**: Tests if a value is within a range of values.

- **IN**: Tests if a value matches any value in a list of values.
- **LIKE**: Used for pattern matching in strings.

Example of Logical Operations:

AND Operator: This query returns employees whose salary is greater than 50,000 **and** who work in the "Sales" department.

```
SELECT first_name, last_name, salary, department
FROM employees
WHERE salary > 50000 AND department = 'Sales';
```

OR Operator: This query returns employees who are in either the "Sales" or "Marketing" departments.

```
SELECT first_name, last_name, salary, department
FROM employees
WHERE department = 'Sales' OR department = 'Marketing';
```

NOT Operator: This query returns employees who do **not** work in the "HR" department.

```
SELECT first_name, last_name, department
FROM employees WHERE NOT department = 'HR';
```

BETWEEN Operator: This query returns employees who are in either the "Sales", "Marketing", or "Finance" departments.

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary BETWEEN 40000 AND 60000;
```

IN Operator:

```
SELECT first_name, last_name, department
FROM employees
WHERE department IN ('Sales', 'Marketing', 'Finance');
```

LIKE Operator (for pattern matching): This query returns employees whose first name starts with the letter "J". The % symbol is a wildcard that matches zero or more characters.

```
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'J%';
```

<u>Combining Arithmetic and Logical Operations</u>

You can combine both arithmetic and logical operations in a single SQL query to perform more complex queries.

Example:

This query calculates a 10% bonus (salary * 0.1) for employees in the "Sales" department who have a salary greater than 50,000.

```
SELECT first_name, last_name, salary, (salary * 0.1) AS bonus
FROM employees
WHERE salary > 50000 AND department = 'Sales';
```

Example 2:

```
SELECT product_name, price, (price * 0.15) AS tax
FROM products
WHERE price BETWEEN 50 AND 200 AND price * 0.15 > 10;
```

<u>3.3 SQL FUNCTIONS [DATE,TIME ,NUMERIC ,STRING Conversion] :</u>

- In DBMS (Database Management System), SQL functions are predefined operations that perform specific tasks on data stored in a database. They help simplify complex queries by allowing you to perform calculations, format data, and manipulate values easily.

- SQL functions are built-in functions in SQL used to perform operations on data such as calculations, string manipulation, date/time handling, and more.
- **Types of SQL Functions:**SQL functions are mainly categorized into two types:

1. Single Row Functions (Scalar Functions):

These functions work on a single row and return one result per row.

Single row functions can be further classified as

i. String Functions: Work for string Data type.
ii. Numeric Functions: Work for Number Data type.
iii. Conversion Functions: Work for Conversion of one data type to another.
iv. Date Functions: Work for Date Data type.
v. Miscellaneous functions

NOTE: In the following query statements we used a dummy table DUAL. It contains one row and one column of data. We can use dual in arithmetic expressions.

2. Group Functions (Aggregate Functions):

These functions work on multiple rows and return one result for the group.

Group functions are: avg(),min(),max(),sum(),count(),count(*)

i) Date and Time Functions :

Function	Description	Example
SYSDATE	Returns current date and time	SELECT SYSDATE FROM DUAL;
CURRENT_DATE	Returns current date in session time zone	SELECT CURRENT_DATE FROM DUAL;
SYSTIMESTAMP	Returns current timestamp with time zone	SELECT SYSTIMESTAMP FROM DUAL;
EXTRACT()	Extracts part of a date	SELECT EXTRACT(YEAR FROM SYSDATE) FROM DUAL;
MONTHS_BETWEEN()	Returns months between two dates	SELECT MONTHS_BETWEEN(DATE '2025-12-01', SYSDATE) FROM DUAL;
ADD_MONTHS()	Adds months to a date	SELECT ADD_MONTHS(SYSDATE, 3) FROM DUAL;
NEXT_DAY()	Finds next specific weekday	SELECT NEXT_DAY(SYSDATE, 'MONDAY') FROM DUAL;
LAST_DAY()	Returns last day of the month	SELECT LAST_DAY(SYSDATE) FROM DUAL;
TRUNC()	Truncates time portion from date	SELECT TRUNC(SYSDATE) FROM DUAL;

Ex: Select CURRENT_DATE from dual;

Output: CURRENT_DATE
21-APR-2025

SELECT EXTRACT(YEAR FROM NOW()); -- Output: 2025

SELECT EXTRACT(MONTH FROM NOW()); --Output: 4
SELECT EXTRACT(DAY FROM NOW()); -- Output: 21
SELECT ROUND(TO_DATE('26-NOV-21', 'DD-MON-YY'), 'YYYY') FROM dual; Output: 01-JAN-25.
Select TRUNC (sysdate, 'MM') from dual; --Output: 01-APR-25.

ii) Numeric Functions :

Function	Description	Example
ROUND(n[, d])	Rounds number n to d decimals	SELECT ROUND(123.456, 2) FROM DUAL;
TRUNC(n[, d])	Truncates number to d decimals	SELECT TRUNC(123.456, 2) FROM DUAL;
MOD()	Returns remainder	SELECT MOD(10, 3) FROM DUAL;
CEIL()	Rounds up to nearest integer	SELECT CEIL(7.3) FROM DUAL;
FLOOR()	Rounds down to nearest integer	SELECT FLOOR(7.9) FROM DUAL;
ABS()	Returns absolute value	SELECT ABS(-55) FROM DUAL;
POWER(x, y)	Returns x raised to the power y	SELECT POWER(2, 5) FROM DUAL;
SQRT()	Returns square root	SELECT SQRT(25) FROM DUAL;

EX:

1. SELECT ABS(-25) AS Absolute_Value FROM dual;
   Output::25
2. SELECT FLOOR(4.9); -- Output: 4
3. SELECT TRUNCATE(4.567, 2); -- Output: 4.56

iii) String Conversion & String Functions :

Function	Description	Example
CONCAT(a, b)	Concatenates two strings	SELECT CONCAT('Hello', 'World') FROM DUAL;
`		`
LENGTH()	Returns string length	SELECT LENGTH('Oracle') FROM DUAL;
SUBSTR()	Extracts substring	SELECT SUBSTR('Database', 1, 4) FROM DUAL;
INSTR()	Finds position of substring	SELECT INSTR('Database', 'base') FROM DUAL;
UPPER()	Converts to uppercase	SELECT UPPER('oracle') FROM DUAL;
LOWER()	Converts to lowercase	SELECT LOWER('ORACLE') FROM DUAL;
TRIM()	Trims both leading/trailing spaces	SELECT TRIM(' Oracle ') FROM DUAL;
TO_CHAR()	Converts number/date to string	SELECT TO_CHAR(SYSDATE, 'DD-Mon-YYYY') FROM DUAL;
TO_NUMBER()	Converts string to number	SELECT TO_NUMBER('1234.56') FROM DUAL;
TO_DATE()	Converts string to date	SELECT TO_DATE('2025-04-21', 'YYYY-MM-DD') FROM DUAL;

EX:

1. Select INITCAP ('data base management system') from dual;
   OUTPUT: Data Base Management System.
2. select rtrim('computerscience','e') as rtrim1 from dual;
   OUTPUT: computerscienc
3. **select LENGTH ('SQ LANGUAGE') from dual;**

OUTPUT: 11.

iv) Conversion Cheat Sheet :

Date to String	TO_CHAR(date, 'format')
Number to String	TO_CHAR(number)
String to Date	TO_DATE('string', 'format')
String to Number	TO_NUMBER('string')

3.4 CREATING TABLES WITH RELATIONSHIPS :

Creating tables with relationships in a Database Management System (DBMS) involves defining the structure of the tables and establishing relationships between them using **primary keys** and **foreign keys**. These relationships help maintain data integrity and allow for efficient querying of related data.

Types of Relationships in DBMS:

1. **One-to-One Relationship**: Each row in one table is associated with exactly one row in another table.
2. **One-to-Many Relationship**: A row in one table can be associated with multiple rows in another table.
3. **Many-to-Many Relationship**: Multiple rows in one table are associated with multiple rows in another table. This requires a junction table (also called a "bridge table") to resolve the relationship.

Steps to Create Tables with Relationships:

1. **Define Tables**: Decide the tables you need based on the relationships between them.
2. **Define Primary Keys (PK)**: Each table should have a primary key that uniquely identifies each row in the table.
3. **Define Foreign Keys (FK)**: Use foreign keys to establish relationships between tables.
4. **Define Constraints**: Set constraints like NOT NULL, UNIQUE, etc., to maintain data integrity.

1. One-to-One Relationship:

In a one-to-one relationship, each record in Table A is associated with one record in Table B.

Example: *Student and Passport*

- A student can have one passport, and each passport is associated with one student.

Tables:

- **Students**: Contains student information.
- **Passports**: Contains passport information.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(100),
    last_name VARCHAR(100)
);

CREATE TABLE Passports (
    passport_id INT PRIMARY KEY,
    student_id INT UNIQUE,  -- One-to-one relationship with Students
    issue_date DATE,
    expiry_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```

Explanation:

- The student_id is a **primary key** in the Students table.

- The student_id in the Passports table is a **foreign key** that establishes the relationship between the two tables.
- **UNIQUE** constraint on student_id in the Passports table ensures that each student can have only one passport.

2. One-to-Many Relationship

In a one-to-many relationship, a row in Table A can be related to many rows in Table B, but each row in Table B is related to only one row in Table A.

Example: *Department and Employees*

- A department can have multiple employees, but each employee belongs to one department.

Tables:

- **Departments**: Contains department information.
- **Employees**: Contains employee information.

```
CREATE TABLE Departments (
   department_id INT PRIMARY KEY,
   department_name VARCHAR(100)
);

CREATE TABLE Employees (
   employee_id INT PRIMARY KEY,
   first_name VARCHAR(100),
   last_name VARCHAR(100),
   department_id INT,
   FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);
```

Explanation:

- The department_id is the **primary key** in the Departments table.
- The department_id in the Employees table is a **foreign key** that links each employee to a department.
- This establishes a one-to-many relationship where a department can have multiple employees, but each employee is associated with one department.

3. Many-to-Many Relationship

In a many-to-many relationship, multiple rows in Table A are related to multiple rows in Table B. This relationship requires a **junction table** (also called an intermediary or linking table) to resolve it.

Example: *Students and Courses*

- A student can enroll in many courses, and a course can have many students enrolled.

Tables:

- **Students**: Contains student information.
- **Courses**: Contains course information.
- **Student_Courses**: A junction table to represent the many-to-many relationship between students and courses.

```
CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   first_name VARCHAR(100),
   last_name VARCHAR(100)
);
```

```
CREATE TABLE Courses (
   course_id INT PRIMARY KEY,
   course_name VARCHAR(100)
);

CREATE TABLE Student_Courses (
   student_id INT,
   course_id INT,
   enrollment_date DATE,
   PRIMARY KEY (student_id, course_id),
   FOREIGN KEY (student_id) REFERENCES Students(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Explanation:

- The Students table has a **primary key** student_id.
- The Courses table has a **primary key** course_id.
- The Student_Courses table has two **foreign keys**:
  - student_id referencing the Students table.
  - course_id referencing the Courses table.
- The **primary key** for the Student_Courses table is a **composite key** consisting of student_id and course_id. This ensures that each combination of student and course is unique.

4. Additional Constraints for Relationships

- **NOT NULL**: Ensures that a column cannot have a NULL value.
- **UNIQUE**: Ensures that all values in a column are unique.
- **CHECK**: Ensures that all values in a column satisfy a specific condition.
- **DEFAULT**: Provides a default value for a column when no value is specified.

Example with Constraints:

```
CREATE TABLE Employees (
   employee_id INT PRIMARY KEY,
   first_name VARCHAR(100) NOT NULL,
   last_name VARCHAR(100) NOT NULL,
   department_id INT,
   hire_date DATE DEFAULT SYSDATE,
   salary DECIMAL(10, 2) CHECK (salary > 0),
   FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);
```

Explanation:

- **NOT NULL** ensures that first_name and last_name cannot be empty.
- **DEFAULT SYSDATE** automatically assigns the current date to hire_date if no value is provided.
- **CHECK (salary > 0)** ensures that the salary is always greater than zero.

3.5 Implementation of key and Integrity Constraints :

Constraints ensure the accuracy and reliability of the data in the database. They can be applied:

- When creating a table
- When modifying a table with ALTER

1. Primary Key Constraint

- Ensures each row has a unique, non-null identifier.

- Only one **primary key** per table.

 Example:

```
CREATE TABLE students (
   student_id NUMBER PRIMARY KEY,
   name VARCHAR2(100)
);
```

Or:

```
CREATE TABLE students (
   student_id NUMBER,
   name VARCHAR2(100),
   CONSTRAINT pk_student PRIMARY KEY (student_id)
);
```

2. NOT NULL Constraint

- Ensures a column cannot have NULL values.

 Example:

```
CREATE TABLE teachers (
   teacher_id NUMBER PRIMARY KEY,
   name VARCHAR2(100) NOT NULL
);
```

3. UNIQUE Constraint

- Ensures that all values in a column (or combination) are different.

 Example:

```
CREATE TABLE users (
   user_id NUMBER PRIMARY KEY,
   email VARCHAR2(100) UNIQUE
);
```

Or with named constraint:

```
CREATE TABLE users (
   user_id NUMBER,
   email VARCHAR2(100),
   CONSTRAINT unq_email UNIQUE (email)
);
```

4. CHECK Constraint

- Restricts the range of values a column can hold.

 Example:

```
CREATE TABLE employees (
   emp_id NUMBER PRIMARY KEY,
   age NUMBER,
   salary NUMBER,
   CONSTRAINT chk_age CHECK (age >= 18 AND age <= 65),
   CONSTRAINT chk_salary CHECK (salary >= 0)
);
```

5. Foreign Key / Referential Integrity

- Links a column to the primary key of another table.
- Ensures that a value exists in the referenced table.

 Example:

```
CREATE TABLE departments (
```

```
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(100)
);

CREATE TABLE employees (
    emp_id NUMBER PRIMARY KEY,
    name VARCHAR2(100),
    dept_id NUMBER,
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

6. Assertions (Note: Not supported in Oracle)

Oracle **does not support CREATE ASSERTION**, unlike the SQL standard.

 Workaround:

- Use **triggers** to simulate assertions.
- Example: To enforce a constraint across multiple tables, write a BEFORE INSERT or BEFORE UPDATE trigger.

Example using Trigger (simulating assertion):

```
CREATE OR REPLACE TRIGGER check_total_salary
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
DECLARE
    total_salary NUMBER;
BEGIN
    SELECT SUM(salary) INTO total_salary FROM employees;
    IF total_salary + :NEW.salary > 1000000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Total salary cannot exceed 1 million');
    END IF;
END;
```

3.6 NESTED QUERIES :

- ✓ A **nested query** (also known as a **subquery**) is a query embedded inside another query.
- ✓ It is useful when you need to retrieve data that depends on the result of another query.
- ✓ The inner query executes first and its result is used by outer query.
- ✓ Nested queries are often used in the **WHERE**, **HAVING**, **FROM**, or **SELECT** clauses of a SQL statement.

Key characteristics of nested queries:

- Inner query runs before outer query.
- Results of inner query for comparison of input data.
- Nested queries are particularly useful for breaking down complex operations into smaller manageable parts, making it easy to retrieve specific results from database.

Advantages of Nested Queries :

- Simplifies complex queries, enhances flexibility and improves readability.
- Commonly used for filtering data, performing calculations or joining database datasets indirectly.

Types of Nested Queries:

1. **Non-Correlated Subqueries**: The inner query can be executed independently of the outer query.
2. **Correlated Subqueries:** The inner query depends on values from the outer query. It is evaluated once for each row in the outer query.

Correlated Nested Query : The inner query depends on the outer query for its execution for each row processed by the outer query. The operators used in correlated nested queries are exists, not exists.

Basic Structure of a Nested Query:

```
SELECT column1, column2
FROM table1
WHERE column1 = (SELECT column1 FROM table2 WHERE condition);
```

Set comparison operators allow you to compare a value with a set of values returned by a subquery. These operators are particularly useful when you need to check for membership in a set or compare values against a set.

Set Comparison Operators:

- **= ANY / = SOME**: Matches if the value is equal to any of the values returned by the subquery.
- **IN**: Matches if the value is contained in the set returned by the subquery.
- **= ALL**: Matches if the value is equal to all the values returned by the subquery.
- **NOT IN**: Matches if the value is not contained in the set returned by the subquery.
- **> ANY**, **< ANY**, **>= ANY**, **<= ANY**: Matches if the value is greater than, less than, etc., any of the values returned by the subquery.
- Let us consider sailors,reserves and boats tables

Table1: sailors

SID	SNAME	RATING	AGE
22	Dustin	7	45
29	Brutus	1	33
31	Lubber	8	55
32	Andy	8	25
58	Rusty	10	35
64	Horatio	7	35
71	Zorba	10	16
74	Horatio	9	40
85	Art	3	25
95	Bob	3	63

Table2: Boats

BID	BNAME	COLOR
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Table3: Reserves

SID	BID	DAY
22	101	10-Oct-98
22	102	10-Oct-98
22	103	08-Oct-98
22	104	07-Oct-98
31	102	10-Nov-98
31	103	06-Nov-98
31	104	12-Nov-98
64	101	05-Sep-98
64	102	08-Sep-98
74	103	08-Sep-98

ALL:

- ✓ The ALL operator returns true if all of the subquery values meet the conditions.

Example:

select s.sid from sailors s where s.rating >=ALL (select s2.rating from sailors s2);

output:

SID
58
71

ANY:

- ✓ The ANY operator returns true if any of the subquery values meet the conditions.

Example:

select s.sid from sailors s where s.rating > ANY
(select s2.rating from sailors s2 where s2.sname like 'B%b');

Output:

SID
58
71
74
31
32
64
22

IN:

- ✓ The outer query retrieves data only for the values that match those written by the inner query.

Example:

select s.sname from sailors s where s.sid IN (select r.sid from reserves r where r.bid = 103);

output:

SNAME
Dustin
Lubber
Horatio

NOT IN:

- ✓ The outer query retrieves the data only for the values that are not present in the inner query result

Example:

select s.sname from sailors s where s.sid NOT IN (select r.sid from reserves r where r.bid = 103);

output:

SNAME
Brutus
Andy
Rusty

Horatio
Zorba
Art
Bob

EXIST:

- ✓ The exit query tests whether a subquery succeeds at least one row.

Example:

select s.sname from sailors s where EXISTS (select * from reserves r where r.bid = 103 and r.sid = s.sid);

output:

SNAME
Dustin
Lubber
Horatio

NOT EXIST:

- ✓ It checks if the subquery returns no rows.

Example:

```
select s.sname from sailors s where NOT EXISTS
        (select * from reserves r where r.bid = 103 and r.sid = s.sid);
```

Output:

SNAME
Brutus
Andy
Rusty
Horatio
Zorba
Art
Bob

- ✓ Nested queries (subqueries) are powerful SQL constructs that allow for complex filtering, aggregation, and conditional logic.
- ✓ These queries can be either **non-correlated** or **correlated**, and they can also use **set comparison operators** to compare values against a set returned by a sub-query.
- ✓ The examples above demonstrate various practical applications of nested queries, including filtering data, performing calculations, and checking complex conditions.

3.7 Sub Queries

- A subquery or inner query or nested query is a query within another SQL query and embedded within the WHERE clause.
- Subquery is used to return the data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Subqueries can be used with the SELECT, INSERT, UPDATE and DELETE statements along with the operators , = , >= , <= , < , > ,IN, BETWEEN, etc
- Subqueries are a powerful feature in SQL that can simplify complex queries by breaking them into smaller, more manageable pieces.

- What is nested sub-query?

  Ans: sub-query inside another query is called nested sub-query.

Example: Find the names of sailors who have reserved a red boat.

```
SELECT sname FROM Sailors
WHERE sid IN (
                SELECT sid
                FROM Reserves
                WHERE bid IN (
                        SELECT bid
                        FROM Boats
                        WHERE color = 'red'
                )
);
```

Output:

SNAME
Dustin
Lubber
Horatio

- By using techniques like **set membership**, **set comparison**, **tests for empty relations**, and **testing for the absence of duplicate tuples**, you can write more expressive and flexible SQL queries.
  - **Set Membership**: Use IN to filter results based on a set of values.
  - **Set Comparison**: Use operators like ALL, ANY, and comparison operators to compare values with the result set of a subquery.
  - **Test for Empty Relations**: Use EXISTS and NOT EXISTS to check for the presence or absence of rows in a subquery.
  - **Test for Absence of Duplicate Tuples**: Use DISTINCT or correlated subqueries to ensure no duplicates in the results.

  These techniques are essential for working with relational databases, enabling you to build more complex and efficient SQL queries.

  Types of Sub Queries:

  1. **Single-row Subquery: Returns one row**
  2. **Multi-row Subquery: Returns multiple rows**
  3. **Correlated Subquery: Uses values from the outer query**

emp_id	Name	department_id	salary	manager_id
1	Alice	10	6000	3
2	Bob	20	7000	4
3	Charlie	10	8000	NULL
4	David	20	9000	NULL
5	Emma	10	7500	3

1. Single-row Subquery

- Returns one row and one column.
- Used with comparison operators like =, >, <, >=, <=.

Example: Find employees earning more than the lowest salary in the company.

SELECT name, salary FROM emp WHERE salary > (SELECT MIN(salary) FROM emp);

Step 1: Compute Minimum Salary

MIN(salary)=6000

Select min(Salary) from employees

Step 2: Employees Earning More Than 6000

Select name,salary from employees where salary>(select min(Salary) from employees);

Name	Salary
Bob	7000
Charlie	8000
David	9000
Emma	7500

2. Multi-row Subquery

- Returns multiple rows.
- Used with IN, ANY, ALL.

Example: Find employees in departments that have at least one employee earning more than 7500.

SELECT name FROM emp

WHERE department_id IN (SELECT department_id FROM emp WHERE salary > 7500);

Step 1: Find Departments Where Someone Earns More

Than 7500

- Charlie (10) → 8000
- David (20) → 9000
- Departments = {10, 20}

Step 2: Employees from These Departments

Name
Alice
Charlie
Emma
Bob

3. Correlated Subquery

- Uses values from the outer query.
- Subquery runs once for each row in the outer query.

Example: Find employees earning more than the average salary of their department.

SELECT name FROM emp e

WHERE salary > (SELECT AVG(salary) FROM emp WHERE department_id = e.department_id);

Step 1: Compute Average Salary Per Department

- Department 10 → 6000+8000+7500=7166.67
- Department 20 → 7000+9000=8000

Step 2: Employees Who Earn More Than Their Department's Average

Name
Charlie
David

3.8 Grouping:

- ✓ The SQL **GROUP BY** clause is used in collaboration with the SELECT statement to arrange identical data into groups.
- ✓ This GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the having clause.
- ✓ The GROUP BY clause is used to **group rows** that have the same values in specified columns, often used with **aggregate functions** like SUM(), COUNT(), AVG(), MIN(), and MAX().

Syntax:

```
SELECT column1, AGG_FUNC(column2)
FROM table_name
GROUP BY column1;
```

- ✓ Taking Sailors, Boats, Reserves tables

Schema: Sailors (SID, sname, Rating, Age)
Boats (BID, bname, Color)
Reserves(SID, BID, Date)

1. Find the age of youngest Sailor for each Rating level.

select s.rating,min(s.age) as youngest from sailors s group by s.rating;

RATING	YOUNGEST
1	33
8	25
7	35
3	25
10	16
9	40

2. SELECT department_id, AVG(salary)

```
FROM employees
WHERE job_title = 'Engineer'
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

First filters engineers, groups them by department, and then filters only those departments where the **average salary** is more than 50,000

Having:

- The Having clause is used along with the Group by clause.The Having clause can be used to select and reject row groups.
- The format of the Having clause is similar to the Where clause consisting of the keywords Having followed by a search condition.
- The Having clause thus specifies a search condition for groups

Syntax:

```
select ( list of columns, aggregate function) from <tablename>
group by column name
having (condition);
```

1. Find the age of youngest Sailor who is eligible to vote i.e. at least 18 years old for each Rating level with at least 2 such Sailors.

```
select s.rating,min(s.age) as min_age
from sailors s
where s.age >=18
group by s.rating having count(*)>1;
```

output:

RATING	MIN_AGE
8	25
7	35
3	25

Key Differences: WHERE vs HAVING

Clause	Filters...	Used With Aggregates?
WHERE	Before grouping	□ No
HAVING	After grouping	□ Yes

3.9 Aggregate Functions:

Aggregate function:

- It is a function where the values of multiple rows are grouped together, has input on certain criteria to form single value of more significant value.
- They are used in select and having causes.
- Returns a single result on group of rows.
- The aggregate functions are MAX(),MIN(),SUM(),AVG(),COUNT(),COUNT(*).

Aggregate functions are:

- MAX(): It returns the max value in given columns.
- MIN(): It returns the min value in given columns
- SUM(): It returns sum of all numeric values in givencolumn
- AVG(): It returns the average of all values in given.columns.
- COUNT(): It returns the total number of all values in the given column (excluding null values).
- COUNT(*): It returns the total number of all rows in the given table (including null values).

Consider the following table:

emp_id	emp_name	department	salary	age
1	Alice	HR	50000	30
2	Bob	IT	70000	28
3	Charlie	IT	65000	35
4	Diana	HR	52000	40
5	Ethan	Sales	60000	32
6	ABC			

MIN():find minimum value in the given column

SQL> SELECT MIN(salary) from employees;

OUTPUT: 50000

MAX(): find maximum value in the given column

SQL>select max(Salary) from Employees;

OUTPUT: 700000

SUM(): sum of values of given column

SQL>select sum(Salary)from Employees;

OUTPUT:297000

COUNT(): Count number of records in the column

SQL>select COUNT(Salary)from Employees;

OUTPUT:5

COUNT(*): Count number of records in the given table including NULL values also

SQL>select COUNT(*)from Employees;

OUTPUT:6

Important Notes:

- Aggregate functions ignore NULL values (except COUNT(*)).
- They return **one row** unless used with GROUP BY.

3.10 Ordering :

- Ordering:-The sql order by clause is used to sort the data ascending or descending order, based on one or more columns.
- The order clause takes ascending order by default unless we mention it.
- We can also sort the data by using column number of selection list

Syntax:

select column-list from tablename order by column....asc /desc;

1. Arranging age of sailors in descending order

Select * from sailor1 order by age DESC;

SID	SNAME	RATING	AGE
95	BOB	3	64
31	LUBBER	8	56
22	DUSTIN	7	45
58	RUSTY	10	35
74	HORATIO	9	35
64	HORATIO	7	35
29	BRUTUS	1	33
32	ANDY	8	26
85	ART	3	25
71	ZORBA	10	16

3. Arranging colors of boats in descending order

select * from boat1 order by color DESC;

BID	BNAME	COLOR
102	INTERLAKE	RED

104	MARINE	RED
103	CLIPPER	GREEN
101	INTERLAKE	BLUE

4. **Arrange the data by using column number:**
   select bname,color from boats order by 1;

BNAME	COLOR
Clipper	**green**
Interlake	**blue**
Interlake	**red**
Marine	**red**

Note: here bname is the first column based on that column the data is arranged

Use Cases of ORDER BY

Use Case	Query Goal
View top 5 salaries	Combine with ROWNUM or FETCH
Alphabetical listings	Sort by name
Chronological order	Sort by date columns
Group-wise top entries	Combine with GROUP BY, ORDER BY, and ROWNUM

Combine with Other Clauses. You can use ORDER BY with:

- WHERE
- GROUP BY
- HAVING
- LIMIT / FETCH FIRST (for top-N queries)

3.11 Joins :

- ✓ SQL joins are essential for combining data from multiple tables in a database, helping to retrieve meaningful information.
- ✓ You can also use join with other clauses, like where, to filter results and get specific data.
- ✓ Allows creating relationships between different sets of data, which is crucial for complex business questions.

Types of joins in SQL:

1. INNER JOIN 2.LEFT JOIN 3.RIGHT JOIN 4.FULL OUTER JOIN
5.SELF JOIN 6.CROSS JOIN

Here assume two tables Class and class_info

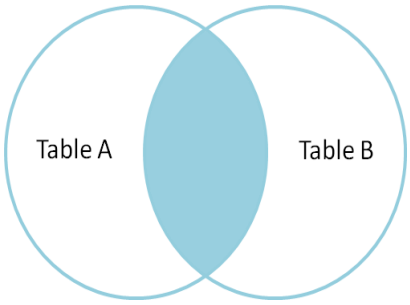
Table1: class

ID	NAME
1	Abhi
2	Adam
4	Alex

Table2: class_info

ID	ADDRESS
1	Delhi
2	Mumbai
3	Chennai

1. INNER JOIN: It retrieves the common data from two tables.



Syntax:

Select col_name from table 1 inner join table 2 on table1.col_name =table2.col_name;

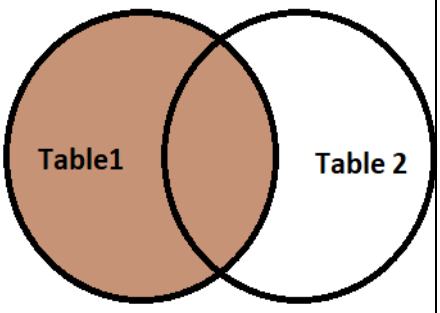
Example:

select * from class inner join class_info on class.id=class_info.id

Output:

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai

2. LEFT JOIN: Returns all rows from left table along with the common data from the tables.



Left Join

Syntax:

Select col_name from table 1 left join table 2 on table 1.col_name=table 2.col_name;

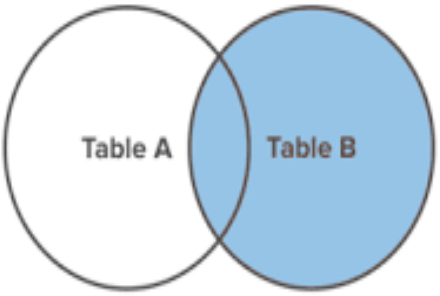
EX:

select * from class left join class_info on class.id=class_info.id

output :

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
4	Alex	-	-

3. RIGHT JOIN: Returns all rows from right table along with the common data from both the tables



SQL RIGHT JOIN

Syntax:

select colname from table1 right join table2 ON table1.colname=table2.colname;

Eg:

select * from class right join class_info on class.id=class_info.id

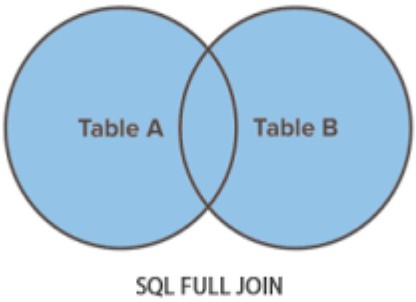
output:

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
-	-	3	Chennai

4. FULL JOIN (Full outer join): Returns all rows when there is a match in either table i.e retrieves data from both the tables

Syntax:

select colname from table1 full outer join table2 on table1.colname=table 2. colname;



Eg:

select * from class full join class_info on class.id=class_info.id;

output:

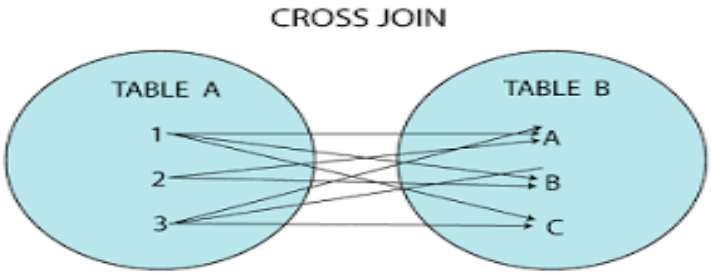
ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
2	Adam	2	Mumbai
4	Alex	-	-
-	-	3	Chennai

5.CROSS JOIN: Returns the cartesian product of both the tables.

Syntax: Select colname from table1 cross join table2;

Example:

select * from class cross join class_info;



output:

ID	NAME	ID	ADDRESS
1	Abhi	1	Delhi
1	Abhi	2	Mumbai
1	Abhi	3	Chennai
2	Adam	1	Delhi
2	Adam	2	Mumbai
2	Adam	3	Chennai
4	Alex	1	Delhi
4	Alex	2	Mumbai
4	Alex	3	Chennai

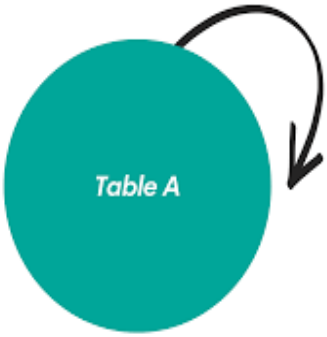
6.SELF JOIN: Regular join, but joins a table with itself.

Syntax:

select table1.colname,table2.colname from table1 join table2 ON table1.colname=table2.colname;

Eg:

select *from class c1 join class c2 on c1.id=c2.id;



output:

ID	NAME	ID	NAME
1	Abhi	1	Abhi
2	Adam	2	Adam
4	Alex	4	Alex

Each type of join in SQL serves a unique purpose:

- **INNER JOIN**: Best when you want only rows that have matching data in both tables.
- **LEFT JOIN**: Best when you need to include all rows from the left table and matching data from the right table (with NULL if no match).
- **RIGHT JOIN**: Best when you need to include all rows from the right table and matching data from the left table (with NULL if no match).

- **FULL OUTER JOIN**: Best when you need all rows from both tables, regardless of whether there is a match.

Join Type	Description	Example SQL
INNER JOIN	Returns rows when there is a match in both tables.	SELECT sailors.sailor_name, boats.boat_name FROM sailors INNER JOIN reserves ...
LEFT JOIN	Returns all rows from the left table, and matching rows from the right table. If no match, NULL for right table.	SELECT sailors.sailor_name, boats.boat_name FROM sailors LEFT JOIN reserves ...
RIGHT JOIN	Returns all rows from the right table, and matching rows from the left table. If no match, NULL for left table.	SELECT sailors.sailor_name, boats.boat_name FROM boats RIGHT JOIN reserves ...
FULL OUTER JOIN	Returns all rows from both tables, with NULL where no match exists.	SELECT sailors.sailor_name, boats.boat_name FROM sailors FULL OUTER JOIN reserves ...

Ex:

1. Find all sailors who have reserved a boat.

```
SELECT sailors.sailor_name, boats.boat_name, reserves.reservation_date
FROM sailors
INNER JOIN reserves ON sailors.sailor_id = reserves.sailor_id
INNER JOIN boats ON reserves.boat_id = boats.boat_id;
```

2. Find all sailors who have reserved a boat.

```
SELECT sailors.sailor_name, boats.boat_name, reserves.reservation_date
FROM sailors
INNER JOIN reserves ON sailors.sailor_id = reserves.sailor_id
INNER JOIN boats ON reserves.boat_id = boats.boat_id;
```

3. Find all boats and the sailors who have reserved them (if any). Include boats that have not been reserved by any sailors.

```
SELECT sailors.sailor_name, boats.boat_name, reserves.reservation_date
FROM boats
RIGHT JOIN reserves ON boats.boat_id = reserves.boat_id
RIGHT JOIN sailors ON reserves.sailor_id = sailors.sailor_id;
```

3.12 Relational Set Operations:

Operation	Description
UNION	Combines results of two queries and removes duplicates
UNION ALL	Combines results and keeps duplicates
INTERSECT	Returns only common rows from both queries
EXCEPT	Returns rows from the first query not in the second
Oracle Note	In Oracle, EXCEPT is called MINUS

Rules for Set Operations

1. Both queries must have **the same number of columns**.
2. Corresponding columns must have **compatible data types**.
3. Column aliases are taken from the **first query**

   Types of set Operators

   1) UNION

   2)UNION ALL

3) INTERSECT
4)SETDIFFERENCE (MINUS)
UNION(U):

- Combines the results of two Tables
- Removes duplicate rows
- Both tables must have the same number of columns with the same datatypes.

Syntax:

select list of columns from <table_name1>
UNION
select list of columns from <table_name2>;

Table A –Employee 1

DEPARTMENT_ID	NAME
1	ALICE
2	BOB
3	CHARLIE

Table B – Employee 2

DEPARTMENT_ID	NAME
3	CHARLIE
4	DAVID
5	EVE

EX:

```
select name from Emplyees1
UNION
select name from Employees2;
```

output:

NAME
ALICE
BOB
CHARLIE
DAVID
EVE

UNION ALL:-

- ✓ Similar to UNION but does not remove duplicates.

Syntax:

Select list of columns from <table name1>UNION ALLselect list of columns from <table name2>;

Ex:

select name from Employees1 UNIONALL select name from Employees2;

output:

NAME
ALICE

BOB
CHARLIE
CHARLIE
DAVID
BOB

INTERSECTION (∩):

- ✓ Returns only the common rows present in both tables

Syntax:

select list of columns from <tablename1>INTERSECT select list of columns from <tablename2>;

Ex:

select name from Employees1 INTERSECT select name from Employees2;

output:

NAME
CHARLIE

Set Difference/Minus (-):

- ✓ Returns rows from the first table that are not present in the second table

Syntax:

select list of columns from <tablename1> MINUS select list of columns from <table name2>;

Ex:

select name from Employees1 MINUS select name from Employees2;

output:

NAME
ALICE
BOB